

Image Processing in PYTHON

Prof. Dr. Aldo André Díaz-Salazar

1 Introduction

During this tutorial, the goal is to become familiar with Python and the NumPy library. You should also get a better feeling for how images are represented as matrices as well as the connection between mathematical expressions and the Python code to implement them. We additionally provide a guide for how to get started with using an IDE for python, which will make your life much easier during the projects.

1.1 Exercises

1. Make sure that youve downloaded the notebook for this lesson at SIGAA:

`INF0417_python-tutorial.ipynb`

2. Start a Jupyter notebook from a terminal:

`jupyter notebook`

3. Find the correct notebook and open it.
4. Follow the instructions in there.

2 Software tools

This section covers unit tests, and development tools. None of this is strictly part of the course, but they are very useful.

2.1 Development tools

There exist a large number of different development tools for Python, ranging from a text-editor like `atom` that can make it easy to get started, to full IDEs, such as `pycharm`. Getting started using an IDE such as `pycharm` might require some time to get familiar with the program, but it will usually save time in the long run due to the very useful auto-complete functions and integrated debugging functionality.

For the projects we strongly recommend that you use `pycharm` or `vscode`.

- **Task 1:** Open `vscode`. Try creating a python file and running it.
- **Task 2:** Open `pycharm`. Try creating a python file and running it.

Both these IDEs will provide you with a good development environment. Choose whichever you feel more comfortable with.

2.2 Unit tests

When writing programs larger than a few lines it is often useful to test the intermediate steps of the program in isolation. There are (at least) two reasons for this:

1. It helps you define what each part of the program should actually do.
2. It makes sure that it actually does what you intended in isolation from the other parts.

For larger projects involving multiple people, **unit tests** can allow you to test each component in isolation for some simple cases. A unit-test is a function that calls another function you want to test with the arguments that should produce some known output, for example:

```
1 def test_addition():
2     a = 10
3     b = 15
4
5     c = 25
6
7     assert a + b == c
```

This function tests that the addition operation behaves as expected. Often you would write multiple tests so that you can be sure that edge cases are handled correctly. A more complicated example for a function that projects a point in an image:

```
1 def project_point_in_image(point, camera_matrix):
2     image_point = camera_matrix @ point
3
4     return image_point
```

With the tests being:

```
1 def project_camera_position():
2     # Project a point that is at the same position as the camera into the image
3     pass
4
5 def project_behind_camera():
6     # Project a point that is behind the camera into the image
7     pass
8
9 def project_point():
10    # Project a sensible point that is in front of the camera into the image
11    pass
```

Try to implement the missing test cases.

- **Hint:** Use the same camera matrix in all tests, but different points.

Note that the point of this exercise is as much to implement the tests as it is to think about what the result should be in each case.